Contents

1 Basic components and combinatorial circuits 1
1.1 The three basic logic gates 1
1.2 Other logic gates 4
1.3 Physical realisation of gates 5
1.3.1 MOSFET transistors 5
1.3.2 CMOS gates 6
1.3.3 Switching delays 7
1.3.4 Moore's law 8
1.4 Algebraic manipulation and duality 11
1.5 Two-layer circuits 14
1.6 Karnaugh maps 17
1.7 Functional completeness of the nand gate 20
1.8 Multiplexers 21
1.9 Summary 22

2 Numbers, basic circuits, and the ALU 23
2.1 Representation of unsigned numbers 23
2.2 Two's complement representation of integers 27
2.3 Adding unsigned numbers 30
2.4 Adding two's complement numbers 33
2.5 Subtraction 35
2.6 Comparing unsigned and two's complement numbers 36
2.7 Arithmetic circuits: addition and subtraction 38
2.7.1 Addition: the half- and full adder 38
2.7.2 Subtraction 39
2.7.3 The carry look-ahead adder 40
2.8 The arithmetic logic unit (ALU) 42
2.9 Multiplication 45
2.10 Alternative representations for numbers 46
2.10.1 Sign and magnitude 47
2.10.2 One's complement 47

2.10.3 Floating-point numbers 48
2.10.4 Parity bits and Hamming codes. 48
2.10.5 Gray code 48
2.11 Representation of character sets 50
2.12 Summary 51

3 Sequential circuits 53
3.1 A one-time latch 53
3.2 The set-reset flip-flop/set-reset latch 54
3.3 The D-latch/D-flip-flop 55
3.4 Registers 59
3.5 Finite state machines 60
3.5.1 An example state machine with four states 61
3.5.2 Encoding the state machine 61
3.5.3 Realising the state machine using logic gates and flip-flops . 63
3.6 Random access memory 67
3.7 Finite state machines to control registers 68
3.8 Hardware description languages 70
3.9 Summary 71

4 An elementary processor 73
4.1 The general structure of the processor 74
4.2 The instruction set 77
4.3 The instruction fetch and the register transfer language 81
4.4 The format of machine code instructions 83
4.5 Implementing instructions on the processor 85
4.6 Optimisation of the execution of instructions 88
4.7 More advanced instructions 91
4.8 Input and output 93
4.9 Interrupts 96
4.10 Summary 97

5 Assembly programming 99
5.1 Labels and comments, EQU and CONS 99
5.2 Arithmetic calculations 102
5.3 A timed loop 104
5.4 Basic data structures 105
5.4.1 Arrays 105
5.4.2 Stacks 106
5.4.3 Linked lists 109
5.5 Memory layout 110
5.5.1 Allocation dependence 111
5.5.2 Relocatable code and data 112
5.6 Subroutines 112
5.6.1 Saving the return address 113

5.6.2 Returning values . . . 115
5.6.3 Passing arguments on the stack . . . 116
5.6.4 Local variables . . . 118
5.7 Interrupt routines . . . 120
5.7.1 Interrupt handlers . . . 120
5.7.2 Installing handlers . . . 122
5.7.3 An example: displaying keyboard strokes . . . 122
5.8 Multitasking and multithreading . . . 125
5.8.1 Timer interrupts and context switching . . . 125
5.8.2 Data structures for multitasking . . . 126
5.9 Summary . . . 129

6 Compiling higher-level languages . . . 131
6.1 A simple higher-level programming language . . . 132
6.2 Context free grammars and parsing . . . 134
6.3 Type checking . . . 136
6.4 Compilation scheme . . . 137
6.5 Compiler optimisation . . . 143
6.6 Compilation of other language constructs . . . 144
6.6.1 Input/output . . . 145
6.6.2 More complex data types . . . 145
6.6.3 Parameter passing . . . 145
6.6.4 Classes and objects . . . 147
6.6.5 Flow control . . . 147
6.6.6 Exception handling . . . 147
6.7 Summary . . . 148

7 Computer organisation . . . 149
7.1 Starting a computer system . . . 149
7.1.1 The Basic Input Output System and the Power On Self Test . 150
7.1.2 The boot loader . . . 151
7.1.3 Unified Extensible Firmware Interface . . . 152
7.2 Operating systems . . . 152
7.2.1 Processor modes . . . 154
7.2.2 System calls . . . 156
7.3 Memory organisation . . . 158
7.3.1 Virtual memory . . . 159
7.3.2 Replacement policies . . . 161
7.3.3 Translation look aside buffers . . . 162
7.3.4 Code, stack, data and other segments . . . 162
7.4 Caches . . . 163
7.4.1 Placement policies . . . 164
7.5 Multi- and many-core processor machines . . . 166
7.6 Summary . . . 168

8 The Raspberry Pi and the ARM processor 169
8.1 Raspberry Pi overview 169
8.2 The ARM architecture 171
8.2.1 ARM architecture instruction sets 172
8.2.2 ARM architecture profiles 173
8.2.3 ARM security modes 174
8.3 Virtual memory (the memory management unit) 174
8.3.1 Memory attributes 175
8.3.2 Memory attributes and the VMMU 176
8.3.3 The system memory management unit 176
8.4 The ARM instruction set 178
8.4.1 Instruction groups 180
8.4.2 Setting flags and conditional execution 184
8.4.3 Arguments and addressing modes 186
8.5 The ARM calling convention 191
8.6 The use of system calls 192
8.7 Summary 193

A An extended instruction set for the simple processor 195

B The ARM 32-bit instruction set 197

C Syntax of the register transfer language 201

Answers to the exercises 203
D.1 Answers for Chapter 1 203
D.2 Answers for Chapter 2 210
D.3 Answers for Chapter 3 216
D.4 Answers for Chapter 4 224
D.5 Answers for Chapter 5 231
D.6 Answers for Chapter 6 240
D.7 Answers for Chapter 7 244
D.8 Answers for Chapter 8 245

References 247

Index 249